

Dizionari avanzati

Dizionari avanzati

Tecniche avanzate per lavorare con i dizionari in Python.

Perché ti servono i dizionari avanzati

I dizionari di base li conosci già. Queste tecniche avanzate ti permettono di **crearli in modo più compatto**, gestire chiavi mancanti senza errori, contare occorrenze, e molto altro — cose che tornano utili continuamente nella programmazione reale.

Dictionary comprehension: creare dizionari in una riga

Esattamente come le list comprehension, ma il risultato è un dizionario:

```
# Crea un dizionario {numero: suo_quadrato} per i numeri da 1 a 5
quadrati = {x: x**2 for x in range(1, 6)}
print(quadrati)    # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Crea un dizionario da due liste
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]
registro = {nome: voto for nome, voto in zip(nomi, voti)}
print(registro)    # {'Alice': 8, 'Bob': 7, 'Carlo': 9}

# Con un filtro: solo i pari e i loro quadrati
pari_quadrati = {x: x**2 for x in range(10) if x % 2 == 0}
print(pari_quadrati)    # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

get() : accedere senza rischio di errore

Se accedi a una chiave che non esiste con `dizionario["chiave"]`, Python lancia un `KeyError`. Il metodo `get()` restituisce invece un valore di default.

```
studenti = {"Alice": 8, "Bob": 7}

print(studenti.get("Alice"))      # 8      ← chiave trovata
print(studenti.get("Carlo"))      # None    ← chiave non trovata, nessun
errore
print(studenti.get("Carlo", "N/A")) # N/A    ← chiave non trovata, usa il
valore di default
```

setdefault() : aggiungi una chiave solo se non esiste

Restituisce il valore se la chiave c'è già, altrimenti la aggiunge con il valore di default e restituisce quello.

```
dizionario = {"a": 1}

valore = dizionario.setdefault("a", 99)  # "a" esiste già → non cambia
nulla
print(valore)      # 1
print(dizionario)  # {'a': 1}

valore = dizionario.setdefault("b", 99)  # "b" non esiste → la aggiunge
print(valore)      # 99
print(dizionario)  # {'a': 1, 'b': 99}
```

Un uso pratico: costruire un dizionario dove i valori sono liste, senza dover controllare se la chiave esiste già:

```
# Raggruppa le parole per la loro prima lettera
parole = ["mela", "banana", "more", "mandarino"]
raggruppate = {}

for parola in parole:
    lettera = parola[0]
    raggruppate.setdefault(lettera, []).append(parola) # Se la chiave non
    c'è, crea una lista vuota

print(raggruppate)
# {'m': ['mela', 'more', 'mandarino'], 'b': ['banana']}
```

Counter : conta le occorrenze automaticamente

Dal modulo `collections`, `Counter` è un dizionario speciale pensato per contare quante volte appare ogni elemento.

```
from collections import Counter

# Conta quante volte appare ogni frutto
frutta = ["mela", "banana", "mela", "uva", "banana", "mela"]
contatore = Counter(frutta)
print(contatore)
# Counter({'mela': 3, 'banana': 2, 'uva': 1})

print(contatore["mela"])      # 3 - occorrenze di "mela"
print(contatore["kiwi"])      # 0 - chiave non trovata → 0 (non
errore!)
print(contatore.most_common(2)) # [('mela', 3), ('banana', 2)] - i 2 più
frequenti

# Conta le lettere in una stringa
lettere = Counter("mississippi")
print(lettere)
# Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
```

defaultdict : valori di default automatici

Un altro strumento di `collections`. Con `defaultdict` non devi mai preoccuparti di controllare se una chiave esiste: se non c'è, viene creata automaticamente con il valore di default che hai scelto.

```
from collections import defaultdict

# defaultdict(list) crea automaticamente una lista vuota per le chiavi
nuove
gruppi = defaultdict(list)

studenti = [("Alice", "3A"), ("Bob", "3B"), ("Carlo", "3A")]
for nome, classe in studenti:
    gruppi[classe].append(nome)  # Non serve controllare se "3A" esiste
    già!

print(dict(gruppi))
# {'3A': ['Alice', 'Carlo'], '3B': ['Bob']}
```

Unire due dizionari

```
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}

# Python 3.9+: operatore | (come l'unione degli insiemi)
unito = d1 | d2
print(unito)  # {'a': 1, 'b': 3, 'c': 4} ← "b" di d2 sovrascrive "b" di d1

# Alternativa (funziona in tutte le versioni)
unito2 = {**d1, **d2}
print(unito2)  # {'a': 1, 'b': 3, 'c': 4}
```

Invertire un dizionario (chiavi ↔ valori)

```
originale = {"a": 1, "b": 2, "c": 3}
invertito = {v: k for k, v in originale.items()} # Scambia chiavi e valori
print(invertito) # {1: 'a', 2: 'b', 3: 'c'}
```

Attenzione: funziona solo se i valori sono unici. Se due chiavi avevano lo stesso valore, una verrà sovrascritta.

Ordinare un dizionario

I dizionari in Python 3.7+ mantengono l'ordine di inserimento. Ma se vuoi ordinarli per chiave o per valore:

```
voti = {"Alice": 8, "Carlo": 9, "Bob": 7}

# Ordina per chiave (alfabetico)
per_nome = dict(sorted(voti.items()))
print(per_nome) # {'Alice': 8, 'Bob': 7, 'Carlo': 9}

# Ordina per valore, dal più alto al più basso
per_voto = dict(sorted(voti.items(), key=lambda x: x[1], reverse=True))
print(per_voto) # {'Carlo': 9, 'Alice': 8, 'Bob': 7}
```